

K. Amruthavalli - 2520090070

Python Flask Tutorial

```
amrutha@Amruthas-MacBook-Air ~ % python3 --version
Python 3.14.6
amrutha@Amruthas-MacBook-Air ~ % mkdir flask_practical
amrutha@Amruthas-MacBook-Air ~ % cd flask_practical
amrutha@Amruthas-MacBook-Air flask_practical % python3 -m pip install virtualenv
Collecting virtualenv
  Downloading virtualenv-21.7.10-py3-none-any.whl.metadata (3.5 kB)
Collecting distlib<1,>=0.3.7 (from virtualenv)
  Downloading distlib-0.4.3-py2.py3-none-any.whl.metadata (5.3 kB)
Collecting filelock<4,>=3.24.2 (from virtualenv)
  Downloading filelock-3.32.7-py3-none-any.whl.metadata (2.0 kB)
Collecting platformdirs<5,>=3.9.1 (from virtualenv)
  Downloading platformdirs-4.11.8-py3-none-any.whl.metadata (5.5 kB)
Collecting python-discovery>=1.6 (from virtualenv)
  Downloading python_discovery-1.6.0-py3-none-any.whl.metadata (5.1 kB)
Downloading virtualenv-21.7.10-py3-none-any.whl (5.3 MB)
 5.3/5.3 MB 4.8 MB/s 0:00:01
Downloading distlib-0.4.3-py2.py3-none-any.whl (470 kB)
Downloading filelock-3.32.7-py3-none-any.whl (100 kB)
Downloading platformdirs-4.11.8-py3-none-any.whl (24 kB)
Downloading python_discovery-1.6.0-py3-none-any.whl (38 kB)
Installing collected packages: distlib, platformdirs, filelock, python-discovery, virtualenv
Successfully installed distlib-0.4.3 filelock-3.32.7 platformdirs-4.11.8 python-discovery-1.6.0 virtualenv-21.7.10
```

Fig 1: Installation of virtualenv for creating a Python virtual environment.

```
amrutha@Amruthas-MacBook-Air flask_practical % virtualenv venv
created virtual environment CPython3.14.6.final.0-64-arm64 in 189ms
creator CPython3Posix(dest=/Users/amrutha/flask_practical/venv, clear=False, no_vcs_ignore=False, global=False)
seeder FromAppData(download=False, pip=bundle, via=copy, app_data_dir=/Users/amrutha/Library/Caches/virtualenv)
added seed packages: pip==26.2.1
activators BashActivator,CShellActivator,FishActivator,NushellActivator,PowerShellActivator,PythonActivator,XonshActivator
```

Fig 2: Creation of virtual environment using virtualenv.

```
amrutha@Amruthas-MacBook-Air ~ % cd flask_practical
amrutha@Amruthas-MacBook-Air flask_practical % source venv/bin/activate
(venv) amrutha@Amruthas-MacBook-Air flask_practical %
```

Fig 3: Activation of the Python virtual environment for Flask development.

```
(venv) amrutha@Amruthas-MacBook-Air flask_practical % pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.5.0-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Downloading markupsafe-3.0.3-cp314-cp314-macosx_11_0_arm64.whl.metadata (2.7 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.5.0-py3-none-any.whl (125 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading markupsafe-3.0.3-cp314-cp314-macosx_11_0_arm64.whl (12 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.5.0 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3
Python 3.14.6 (v3.14.6:c63aec69bd5, Jun 10 2026, 08:07:54) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import flask
>>> exit
```

Fig 4: Installation of Flask framework inside the virtual environment.

```
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3
Python 3.14.6 (v3.14.6:c63aec69bd5, Jun 10 2026, 08:07:54) [Clang 21.0.0 (clang-2100.1.1.101)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> import flask
>>> 
```

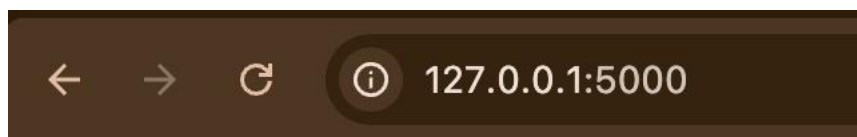
Fig 5: Verification of successful Flask installation using the Python shell.

```
script.py ×
venv > script.py
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/')
6  def home():
7      return "hello, this is our first flask website"
8
9  if __name__ == '__main__':
10     app.run(debug=True)
```

Fig 6: Python code for creating the first Flask web application.

```
amrutha@Amruthas-MacBook-Air flask_practical % source /Users/amrutha/flask_practical/venv/bin/activate
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3 venv/script.py
* Serving Flask app 'script'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server in
stead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 869-871-130
```

Fig 7: Successful execution of the first Flask application on the local development server.



hello, this is our first flask website

Fig 8: Output of the first Flask application displayed in the web browser.

```
script.py X
venv > script.py > ...
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/home')
6  def home():
7      return "hello, welcome to our website"
8
9  if __name__ == "__main__":
10     app.run(debug=True)
```

Fig 9: Flask app routing using the /home route.

```
amrutha@Amruthas-MacBook-Air flask_practical % source /Users/amrutha/flask_practical/venv/bin/activate
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3 venv/script.py
* Serving Flask app 'script'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI se
rver instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 869-871-130
```

Fig 10: Execution of the Flask application with the /home route.



hello, welcome to our website

Fig 11: Output of the Flask /home route displayed in the browser.

```
script.py X
venv > script.py > ...
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/home/<name>')
6  def home(name):
7      return "hello," + name
8
9  if __name__ == "__main__":
10     app.run(debug=True)
```

Fig 12: Flask app routing using a variable URL parameter.

```
amrutha@Amruthas-MacBook-Air flask_practical % source /Users/amrutha/flask_practical/venv/bin/activate
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3 venv/script.py
* Serving Flask app 'script'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI se
rver instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 869-871-130
127.0.0.1 - - [16/Sep/2026 09:39:14] "GET /home/amrutha HTTP/1.1" 200 -
```

Fig 13: Execution of Flask application with a variable URL route.



hello,amrutha

Fig 14: Output of Flask variable URL routing displayed in the browser.

```
script.py X

venv > script.py > ...
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/home/<name>')
6  def home(name):
7      return "hello," + name
8
9  if __name__ == "__main__":
10     app.run(debug=True, port=5001)
```

Fig 15: Flask app routing using an integer URL converter.

```
amrutha@Amruthas-MacBook-Air flask_practical % source /Users/amrutha/flask_practical/venv/bin/activate
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3 venv/script.py
* Serving Flask app 'script'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI se
rver instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 869-871-130
127.0.0.1 - - [16/Sep/2026 09:42:57] "GET /home/21 HTTP/1.1" 200 -
127.0.0.1 - - [16/Sep/2026 09:42:57] "GET /favicon.ico HTTP/1.1" 404 -
█
```

Fig 16: Execution of Flask application using an integer URL converter

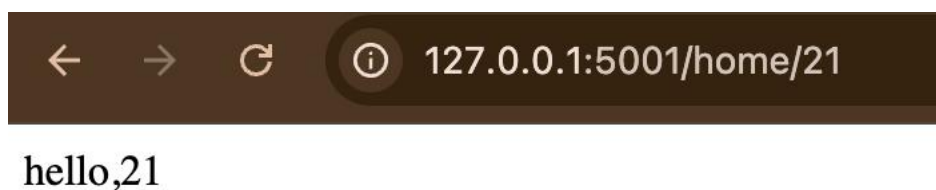


Fig 17: Output of Flask integer URL routing displayed in the brows

```
script.py X
venv > script.py > ...
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  def about():
6      return "This is about page"
7
8  app.add_url_rule("/about", "about", about)
9
10 if __name__ == "__main__":
11     app.run(debug=True, port=5001)
```

Fig 18: Flask routing using the `add_url_rule()` function.

```
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3 venv/script.py
* Serving Flask app 'script'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI se
rver instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 869-871-130
127.0.0.1 - - [16/Sep/2026 09:50:48] "GET /about HTTP/1.1" 200 -
```

Fig 19: Execution of Flask application using an integer URL converter



This is about page

Fig 20: Output of Flask integer URL routing displayed in the brows

```
script.py X
venv > script.py > ...
13 @app.route('/student')
14 def student():
15     return 'student'
16
17 @app.route('/user/<name>')
18 def user(name):
19     if name == 'admin':
20         return redirect(url_for('admin'))
21     if name == 'librarian':
22         return redirect(url_for('librarian'))
23     if name == 'student':
24         return redirect(url_for('student'))
25
26 if __name__ == '__main__':
27     app.run(debug=True, port=5001)
```

Fig 21: Flask URL building using `url_for()` and `redirect` functions.

```
amrutha@Amruthas-MacBook-Air flask_practical % source /Users/amrutha/flask_practical/venv/bin/activate
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3 venv/script.py
* Serving Flask app 'script'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI se
rver instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 869-871-130
127.0.0.1 - - [16/Sep/2026 10:14:36] "GET /user/admin HTTP/1.1" 302 -
127.0.0.1 - - [16/Sep/2026 10:14:36] "GET /admin HTTP/1.1" 200 -
127.0.0.1 - - [16/Sep/2026 10:14:52] "GET /user/librarian HTTP/1.1" 302 -
127.0.0.1 - - [16/Sep/2026 10:14:52] "GET /librarian HTTP/1.1" 200 -
127.0.0.1 - - [16/Sep/2026 10:15:02] "GET /user/student HTTP/1.1" 302 -
127.0.0.1 - - [16/Sep/2026 10:15:02] "GET /student HTTP/1.1" 200 -
127.0.0.1 - - [16/Sep/2026 10:15:40] "GET /user/admin HTTP/1.1" 302 -
127.0.0.1 - - [16/Sep/2026 10:15:40] "GET /admin HTTP/1.1" 200 -
```

Fig 22: Execution of Flask URL building application using `url_for()` and redirects.



admin

Fig 23: Output of dynamic URL redirection using `url_for()` in Flask.

```
script.py X login.html X
login.html > html
1 <html>
2 <body>
3 <form action="http://127.0.0.1:5001/login" method="post">
4 <table>
5 <tr><td>Name</td>
6 <td><input type="text" name="uname"></td></tr>
7
8 <tr><td>Password</td>
9 <td><input type="password" name="pass"></td></tr>
10
11 <tr><td><input type="submit"></td></tr>
12 </table>
13 </form>
14 </body>
15 </html>
```

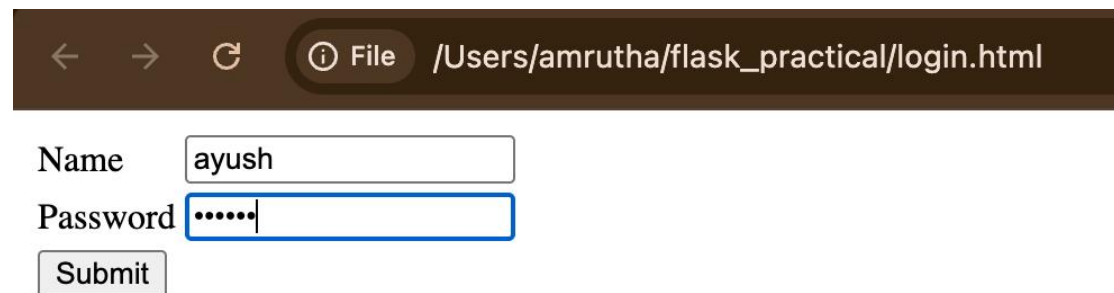
Fig 24: HTML login form created for demonstrating the POST method in Flask.

```
script.py X login.html X post_example.py X
post_example.py > ...
1 from flask import *
2
3 app = Flask(__name__)
4
5 @app.route('/login', methods=['POST'])
6 def login():
7     uname = request.form['uname']
8     passwd = request.form['pass']
9
10     if uname == "ayush" and passwd == "google":
11         return "Welcome %s" % uname
12
13 if __name__ == '__main__':
14     app.run(debug=True, port=5001)
```

Fig 25: Flask POST method program for processing login form data.

```
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3 post_example.py
* Serving Flask app 'post_example'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 869-871-130
```

Fig 26: Execution of the Flask POST method program on the local development server.

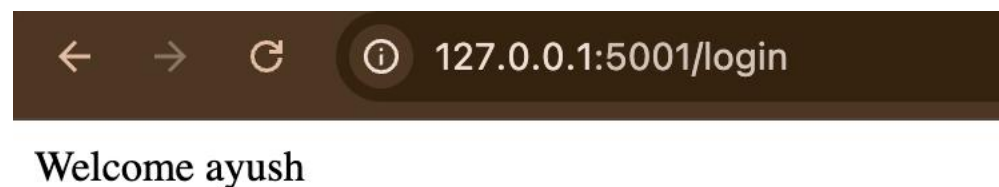


← → ↻ ⓘ File /Users/amrutha/flask_practical/login.html

Name

Password

Fig 27: Login form used to submit user credentials using the POST method.



← → ↻ ⓘ 127.0.0.1:5001/login

Welcome ayush

Fig 28: Successful login output obtained using the Flask POST method.

```
script.py × login.html × get_example.py × post_example.py ×
get_example.py > ...
1  from flask import *
2
3  app = Flask(__name__)
4
5  @app.route('/login', methods=['GET'])
6  def login():
7      uname = request.args.get('uname')
8      passwd = request.args.get('pass')
9
10     if uname == "ayush" and passwd == "google":
11         return "Welcome %s" % uname
12
13 if __name__ == '__main__':
14     app.run(debug=True, port=5001)
```

Fig 29: Flask GET method program for retrieving login form data.

```
(venv) amrutha@Amruthas-MacBook-Air flask_practical % python3 get_example.py
* Serving Flask app 'get_example'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 869-871-130
```

Fig 30: Execution of the Flask GET method program on the local development server.



Welcome ayush

Fig 31: Successful GET request output showing submitted form data in the browser URL.